

Metode Formale in Ingineria Software - Info an 3 + Mate-Info an 3 + CTI an 4, sem 2

An univ. 2025-2026

Curs: Florentin Ipate

Laborator: Ana Iova

Invited speaker: Marian Gheorghe (University of Bradford).

- I. **Objective:** Asimilarea principalelor concepte si tehnici de formal modelling and verification, model-based testing, membrane computing si property-based testing.

II. Cerinte

Notarea se va face pe baza de proiect. Proiectele se vor efectua in echipe (dimensiunea maxima a echipei pentru fiecare tema este specificata mai jos) si vor fi prezentate la curs sau la laborator, in functie de tema aleasa (conform informațiilor de mai jos), in saptamanile 5-10, conform programarii stabilite cu cadrul didactic de curs/laborator, astfel incat numarul de studenti care prezinta in cadrul unui curs/laborator sa nu depaseasca 10. Tema proiectului va fi aleasa din lista de mai jos. Fiecare tema din lista poate fi aleasa de cel mult 2 echipe.

Prezentarea va fi sub forma de slide-uri, insotite de demo-uri. La prezentare este necesara prezenta intregii echipe, fiecare student descriind principala sa contributie la proiect. Timpul alocat fiecarui student este de 10 minute. Atunci cand echipa considera ca nu toti membrii echipei au avut o contributie egala la realizarea proiectului, se va indica, in procente, contributia estimata a fiecaruia.

III. Teme proiecte

A. **Formal Modelling and Verification** – prezentarea acestor teme se va face la curs

1. Modelare si analiza cu Event-B, Rodin

- Prezentati capabilitatile de modelare si analiza ale limbajului Event-B: evenimente, rafinare, demonstrare automata a proprietatilor.
- Ilustrati aceste capabilitati folosind unul sau mai multe exemple create de echipa.

Marime maxima echipa: 3 studenti

Bibliografie: [1,2]

2. Model checking cu Event-B, Rodin si Pro B

- Prezentati abordarea de model checking folosind Event-B, Rodin si Pro B descrisa in [3].
- Ilustrati aceasta abordare folosind unul sau mai multe exemple create de echipa (diferite de exemplul din articolul [3]).

Marime maxima echipa: 3 studenti

Bibliografie: [3,4]

3. Model checking cu NuSMV

- Prezentati pe scurt logica LTL si logica CTL.

- Ilustrați capabilitățile de model checking (verificare de proprietăți LTL și CTL) folosind unul sau mai multe exemple create de echipă.

Marime maxima echipa: 3 studenti

Bibliografie: [5,6]

B. Model-based Testing – prezentarea acestor teme se va face la curs

4. Model based testing cu JSXM

- Prezentați limbajul de modelare bazat pe stream X-machines precum și principalele facilități oferite de utilitarul JSXM: animarea modelului, generarea testelor pe baza modelului de forma stream X-machine, implementarea testelor în JUnit.
- Ilustrați aceste capabilități folosind unul sau mai multe exemple create de echipă.

Marime maxima echipa: 3 studenti

Bibliografie: [7,8,9]

Observație: pentru instalarea tool-ului se va consulta documentul „JSXM installation instructions”.

5. Search based testing pentru EFSM

- Prezentați metoda de generare de date de test prezentată în [10].
- Scrieți un program care implementează metoda de generare de date de test prezentată în [10] și ilustrați funcționarea acesteia pe un exemplu.

Marime maxima echipa: 2 studenti

Bibliografie: [10]

6. GUI testing cu Fastbot2

- Prezentați principalele facilități oferite de utilitarul Fastbot2: construirea unui model probabilistic și strategii de testare ghidată bazată pe model (îmbunătățită printr-un algoritm de reinforcement learning).
- Ilustrați aceste capabilități folosind unul sau mai multe exemple create de echipă.

Marime maxima echipa: 3 studenti

Bibliografie: [11,12]

7. Model based testing cu GraphWalker

- Prezentați principalele facilități oferite de utilitarul GraphWalker: generarea testelor din modelul mașinilor cu stări finite (FSM) folosind algoritmi de traversare precum A* sau parcurgere random cu diverse criterii de acoperire (state, edge, requirement).
- Ilustrați aceste capabilități folosind unul sau mai multe exemple create de echipă.

Marime maxima echipa: 2 studenti

Bibliografie: [13,14]

C. Membrane Computing Applications - prezentarea acestor teme se va face la curs

8. P-Lingua and Mecosim

- Prezentați limbajul P-Lingua și ilustrați folosirea acestuia pe un exemplu creat de echipă.
- Utilizați MeCoSim pentru simularea și analiza exemplului creat anterior.

Marime maxima echipa: 3 studenti

Bibliografie: [15,16,17]

9. Modelling and simulation of tissue P systems

- Prezentați sintaxa limbajului P-Lingua pentru tissue P systems și ilustrați folosirea acestuia pe un exemplu creat de echipă.
- Prezentați intuitiv algoritmul de simulare a tissue P systems și ilustrați funcționarea acestuia pe exemplul dat.

Marime maxima echipa: 3 studenti

Bibliografie: [17,18]

10. Modelling and simulation of spiking neural P systems

- Prezentați sintaxa limbajului P-Lingua pentru spiking neural P systems și ilustrați folosirea acestuia pe un exemplu creat de echipă.
- Prezentați algoritmul de simulare a spiking neural P systems și ilustrați funcționarea acestuia pe exemplul dat.

Marime maxima echipa: 3 studenti

Bibliografie: [17,19]

11. kP Systems si KPWorkbench

- Prezentați modelul kP System și ilustrați acest formalism printr-un exemplu creat de echipă.
- Prezentați facilitățile KPWorkBench (modelare, simulare și verificare formală) pe exemplul anterior.

Marime maxima echipa: 3 studenti

Bibliografie: [20,21,22]

12. Modeling and Verification of P Systems Using Rodin and ProB

- Ilustrați printr-un exemplu reprezentarea în Event-B a unui P system.
- Ilustrați pe exemplul anterior verificarea formală folosind ProB.

Marime maxima echipa: 3 studenti

Bibliografie: [23,24,25]

13. Formal verification of P Systems Using NuSMV

- Ilustrați printr-un exemplu reprezentarea în SMV a unui transformation-communication P system.
- Ilustrați pe exemplul anterior verificarea formală folosind NuSMV.

Marime maxima echipa: 3 studenti

Bibliografie: [26,27]

14. Model checking based test generation from P systems

- Prezentați criteriile de acoperire rule coverage și context-dependent rule coverage și ilustrați aceste tipuri de acoperire cu un exemplu creat de echipă.
- Arătați cum pot fi generate teste pentru criteriile de acoperire mai sus menționate și generați teste pentru exemplul de mai sus folosind NuSMV.

Marime maxima echipa: 3 studenti

Bibliografie: [27,28]

D. Search-based testing - prezentarea acestor teme se va face la laborator

15. Utilizarea algoritmilor evolutivi pentru satisfacerea criteriilor formale de testare software folosind EvoSuite

- Investigati modul în care criterii formale de testare (ex. acoperirea instrucțiunilor și ramurilor) pot fi modelate și satisfăcute automat, folosind algoritmi evolutivi, având EvoSuite ca studiu de caz.
- Prezentați fundamentul teoretic care sta în spatele acestui tool, câteva criterii de testare (ex. Statement coverage și branch coverage), tool-ul EvoSuite, funcții de fitness
- Implementați un mic proiect Java pentru care să utilizați tool-ul EvoSuite pentru generare de teste, analizați rezultatele.

Mărime maximă echipă: 3 studenți

Bibliografie: [31, 32, 33, 34]

16. Generarea automată de teste și date de intrare folosind AVMf și criterii formale de acoperire

- Aplicarea framework-ului **AVMf** (*Alternating Variable Method Framework*) pentru generarea automată de teste și date de intrare, analizând modul în care algoritmul satisface **criterii formale de testare** (statement coverage, branch coverage) și evaluând performanța și limitele metodei.
- Utilizarea **AVMf** pe un proiect Java simplu sau mediu, analizați rezultatele

Mărime maximă echipă: 3 studenți

Bibliografie: [35,36]

17. Generarea automată de scenarii de testare pentru sisteme autonome folosind AmbieGen

- Aplicarea framework-ului **AmbieGen** pentru generarea automată a scenariilor de testare pentru un sistem autonom (ex. robot, simulator de trafic), analizând modul în care algoritmi evolutivi satisfac criterii formale de testare și evaluând performanța și limitele metodei.
- Implementați și prezentați rezultatele

Mărime maximă echipă: 3 studenți

Bibliografie [37,38]

18. Search based testing pentru EFSM - folosind Algoritmi Genetici hibrizi

- Prezentați metoda de generare de date de test prezentată în [29].
- Scrieți un program care implementează metoda de generare de date de test prezentată în [29] și ilustrați funcționarea acesteia pe un exemplu. Comparați rezultatele obținute cu cele ale implementării din articol.

Mărime echipă: 2-3 studenți

Bibliografie: [29]

19. Search based testing pentru EFSM - folosind algoritmi genetici multi-obiectiv

- Prezentați metoda de generare de date de test prezentată în [30].

- Scrieti un program care implementeaza metoda de generare de date de test prezentata in [30] si ilustrati functionarea acesteia pe un exemplu. Comparati rezultatele obtinute cu cele ale implementarii din articol.

Marime echipa: 2-3 studenti

Bibliografie: [30]

E. Metode formale de testare si verificare folosind limbajul Haskell - prezentarea acestor teme se va face la laborator

20. Testarea property-based a modulelor software folosind QuickCheck

- Aplicarea **QuickCheck** pentru generarea automată de teste bazate pe proprietăți (*property-based testing*) pentru un modul software scris in Haskell. Studenții vor defini proprietăți formale și vor analiza modul în care framework-ul generează teste, verificând satisfacerea criteriilor formale și evaluând eficiența metodei.
- Prezentare QuickCheck, Experiment practic, analiza si concluzii.
- Se puncteaza si complexitatea programului testat.

Marimea maxima echipa: 2 studenti

Bibliografie [39,40,41]

21. Definirea și verificarea formală a unui limbaj de programare miniatural folosind Haskell

- Proiectul urmărește: definirea **formală** a unui limbaj de programare simplu, implementarea lui în Haskell, demonstrarea unor **proprietăți teoretice** (determinism, corectitudine semantică).
- Se va defini un limbaj de programare miniatural, de dimensiune redusă, care să conțină **cel puțin**: Variabile și valori (numerice și/sau booleene), Expresii aritmetice și booleene, Instrucțiuni: **skip**, atribuire ($x := e$), secvențiere ($S_1 ; S_2$), structură condițională (**if**), buclă (**while**)
- Limbajul trebuie specificat prin: sintaxă abstractă (AST), descriere formală a componentelor
- Specificarea semanticii formale: cat mai clar si coerent, toate regulile de executie ale programului. Se va implementa interpretorul care va reflecta fidel regulile semanticii definite anterior.
- Proiectul trebuie să includă **verificarea automată a proprietăților limbajului**, folosind **QuickCheck** sau tehnici similare. Definirea a **minimum 3 proprietăți formale**, cum ar fi: determinismul semanticii, **skip** nu modifică starea, execuția secvențială este compozițională.
- Pentru echipa de 4 studenti comportamentul limbajului va fi definit utilizând **semantică big-step**.

Mărime maxima echipa: 3-4 studenti

Bibliografie: [39,40,41, 42] + suport cursurile PF si FLP anul 2.

22. Definirea și verificarea formală a unui limbaj miniatural pentru expresii aritmetice folosind Haskell

- Studenții trebuie să definească și să implementeze un limbaj miniatural de expresii aritmetice și să-i specifice formal semantică. Apoi, se va construi un interpretor în Haskell care să respecte această semantică.
- Se va defini un limbaj care să conțină cel puțin: tipuri de date numerice, constante, variabile, operații aritmetice: +, -, *, /, paranteze pentru priorități: $(e1 + e2) * e3$.
- Limbajul trebuie specificat prin: sintaxă abstractă (AST), descriere formală a componentelor
- Specificarea semanticii formale: cât mai clar și coerent, toate regulile de execuție ale programului. Se va implementa interpretorul care va reflecta fidel regulile semanticii definite anterior.
- Proiectul trebuie să includă **verificarea automată a proprietăților limbajului**, folosind **QuickCheck** sau tehnici similare. Definirea a cât mai multe **proprietăți formale**, cum ar fi: evaluarea constantei returnează valoarea ei, evaluarea unei expresii fără variabile nu depinde de stare, comutativitate, identitate, idempotență, simplificare, etc.
- Pentru echipa de 4 studenți comportamentul limbajului va fi definit utilizând **semantică big-step**.

Mărime maximă echipă: 3-4 studenți

Bibliografie: [39,40,41, 42] + suport cursurile PF și FLP anul 2.

G. Teme suplimentare - de prezentat la laborator

23. Testarea sistemelor cu stare folosind EvoMBT

- Studenții vor analiza și utiliza **EvoMBT** pentru a genera teste automat din modele de tip EFSM, evaluând performanța și eficiența generării de teste.
- Studenții vor alege un sistem cu stări (ex. API CRUD, coadă FIFO, protocol simplu) și îl vor modela sub forma unui EFSM.
- Se va integra modelul în EvoMBT și se va genera suita de teste.
- Se vor analiza rezultatele.

Mărime maximă echipă: 2-3 studenți

Bibliografie: [43,44, 45]

24. Evaluarea generatorului de scenarii *AdaFrenetic* pentru testarea sistemelor Cyber Physical

- Proiectul urmărește analiza și utilizarea tool-ului **AdaFrenetic** pentru generarea automată de scenarii de test pentru sisteme autonome.
- Studenții vor rula cel puțin **5 scenarii generate automat**. Și vor colecta rezultate privind:

- numărul scenariilor valide/ invalide,
- ieșiri din bandă (OBE),
- eficiență / timp de generare.
- Studentii vor realiza (si vor explica) la alegere una dintre modificări:
 - schimbarea funcției de fitness,
 - ajustarea limitelor (viteza, OBE tolerance),
 - schimbarea strategiei de generare a punctelor de drum.

Mărime maxima echipa: 2-3 studenti

Bibliografie: [46, 47]

IV. Bibliografie

- [1]. <http://www.event-b.org/>
- [2]. Jean-Raymond Abrial. Modeling in Event-B: System and Software Engineering
<https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=0861902a5167a3f98b4fe2532f3a83d6aa4a43d5>
- [3]. Adrian Turcanu, Talal Shaikh and Cristina Nicoleta Mazilu, On Model Checking of a Robotic Mechanism. <https://scholars.direct/Articles/robotics/jra-4-018.pdf>
- [4]. <https://www3.hhu.de/stups/prob/>
- [5]. <https://www.cs.utexas.edu/users/moore/acl2/seminar/2010.05-19-krug/slides.pdf>
- [6]. <https://nusmv.fbk.eu/>
- [7]. Florentin Ipatu, Mike Holcombe: Specification and testing using generalized machines: a presentation and a case study, Software Testing, Verification and Reliability, 8, 61-81 (1998) <http://www.ifsoft.ro/~florentin.ipate/publications/STVR98%20-%20Specification%20and%20testing%20using%20generalised%20machines.pdf>
- [8]. <http://www.jsxm.org/>
- [9]. Dimitris Dranidis, Konstantinos Bratanis, Florentin Ipatu: JSXM: a tool for automated test generation, in SEFM, 352-366 (2012).
- [10]. Raluca Lefticaru, Florentin Ipatu: Automatic State-Based Test Generation Using Genetic Algorithms, in SYNASC, 188-195 (2007).
<http://www.ifsoft.ro/~florentin.ipate/publications/SYNASC%202007%20Automatic%20State-Based%20Test%20Generation%20Using%20Genetic%20Algorithms.pdf>
- [11]. https://github.com/bytedance/Fastbot_Android
- [12]. <https://tingsu.github.io/files/ASE22-industry-Fastbot.pdf>
- [13]. <https://graphwalker.github.io/>
- [14]. Muhammad Nouman Zafar, Wasif Afzal, Eduard Enoiu, Athanasios Stratis, Aitor Arrieta, Goiuria Sagardui: Model-Based Testing in Practice: An Industrial Case Study using GraphWalker. ISEC 2021: 5:1-5:11
http://ebiltegia.mondragon.edu/xmlui/bitstream/handle/20.500.11984/5411/Model-Based%20Testing%20in%20Practice_An%20Industrial%20Case%20Study%20using%20GraphWalker.pdf?sequence=1&isAllowed=y
- [15]. A P-Lingua Programming Environment for Membrane Computing. Workshop on Membrane Computing 2008: 187-203.
<https://idus.us.es/bitstream/handle/11441/68600/1/p-lingua-WMC9.pdf?sequence=1>
- [16]. Ignacio Pérez-Hurtado, Luis Valencia-Cabrera, Mario J. Pérez-Jiménez, Maria Angels Colomer, Agustin Riscos-Núñez: MeCoSim: A general purpose software tool for

simulating biological phenomena by means of P systems. BIC-TA 2010: 637-643.

<http://www.p-lingua.org/mecosim/doc/downloads/mecosim.pdf>

- [17]. <https://github.com/RGNC/plingua>
- [18]. Miguel A. Martínez-del-Amor, Ignacio Pérez-Hurtado, Mario J. Pérez-Jiménez, Agustin Riscos-Núñez: A P-Lingua based simulator for tissue P systems. J. Log. Algebraic Methods Program. 79(6): 374-382 (2010)
<https://www.sciencedirect.com/science/article/pii/S1567832610000135/pdf?md5=5145407652ddc0bb0809ce8f3ea46cec&pid=1-s2.0-S1567832610000135-main.pdf>
- [19]. Luis F. Macías-Ramos, Ignacio Pérez-Hurtado, Manuel García-Quismondo, Luis Valencia-Cabrera, Mario J. Pérez-Jiménez, Agustin Riscos-Núñez: A P-Lingua Based Simulator for Spiking Neural P Systems. Int. Conf. on Membrane Computing 2011: 257-281
https://idus.us.es/bitstream/handle/11441/79580/Mac%C3%ADasRamos2012_Chapter_APLinguaBasedSimulatorForSpiki.pdf
- [20]. S. Konur, L. Mierla, F. Ipate, M. Gheorghe: kpworkbench: A software suit for membrane systems, SoftwareX, 11: 100407, 2020.
<https://www.ifsoft.ro/~florentin.ipate/publications/SoftwareXKMIG.pdf>
- [21]. M. Gheorghe, F. Ipate, L. Mierla, S. Konur: kPWorkbench: A Software Framework for Kernel P Systems, BWMC 2015, 179-194, 2015.
https://idus.us.es/xmlui/bitstream/handle/11441/33048/179_bwmc15.pdf?sequence=1&isAllowed=y
- [22]. <https://github.com/Kernel-P-Systems/kPWorkbench>
- [23]. F. Ipate, A. Turcanu: Modeling, Verification and Testing of P Systems Using Rodin and ProB, in Ninth Brainstorming Week on Membrane Computing, 209-220, Fenix Editora, Sevilla, 2011.
<https://www.ifsoft.ro/~florentin.ipate/publications/BWMC%202011%20Verification%20and%20Testing%20of%20P%20Systems%20Using%20Rodin%20and%20ProB.pdf>
- [24]. <http://www.event-b.org/>
- [25]. <https://prob.hhu.de/>
- [26]. M. Gheorghe, F. Ipate, R. Lefticaru, C. Dragomir: An integrated approach to P systems formal verification, in Int. Conf. on Membrane Computing (CMC 2010), Jena, Germany, August 24-27, 2010. Revised Selected Papers, LNCS 6501 Springer, 226-239, 2011.
<https://www.ifsoft.ro/~florentin.ipate/publications/LNCS%20CMC%202010%20An%20Integrated%20Approach%20to%20P%20Systems%20Formal%20Verification.pdf>
- [27]. <https://nusmv.fbk.eu/>
- [28]. R. Lefticaru, F. Ipate, M. Gheorghe: Model Checking Based Test Generation from P Systems Using P-Lingua, Romanian Journal of Information Science and Technology, 13(2), 153-168, 2010.
<https://www.ifsoft.ro/~florentin.ipate/publications/ROMJIST%202010%20Model%20Checking%20Based%20Test%20Generation%20from%20P%20Systems%20Using%20P-Lingua.pdf>
- [29]. A. Turlea, F. Ipate and R. Lefticaru, "A Hybrid Test Generation Approach Based on Extended Finite State Machines," 2016 18th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC), Timisoara, Romania, 2016, pp. 173-180, doi: 10.1109/SYNASC.2016.037.
<https://www.ifsoft.ro/publications/SYNASC2016.pdf>

- [30]. A. Turlea, F. Ipate and R. Lefticaru, "A Test Suite Generation Approach Based on EFSMs Using a Multi-objective Genetic Algorithm," 2017 19th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC), Timisoara, Romania, 2017, pp. 153-160, doi: 10.1109/SYNASC.2017.00034.
<https://www.ifsoft.ro/publications/SYNASC2017.pdf>
- [31]. G. Fraser and A. Arcuri, "EvoSuite: Automatic Test Suite Generation for Object-Oriented Software," *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, 2013, pp. 416–426. <https://www.evosuite.org/wp-content/papercite-data/pdf/eseconfse11.pdf>
- [32]. G. Fraser and A. Arcuri, "Whole Test Suite Generation," *IEEE Transactions on Software Engineering*, vol. 39, no. 2, pp. 276–291, 2013. https://www.evosuite.org/wp-content/papercite-data/pdf/tse12_evosuite.pdf
- [33]. EvoSuite Official Documentation. [Online]. Available: <https://www.evosuite.org/documentation/>
- [34]. EvoSuite GitHub Repository. [Online]. Available: <https://github.com/EvoSuite/evosuite>
- [35] P. S. McMinn și G. M. Kapfhammer, "AVMf: An Open-Source Framework and Implementation of the Alternating Variable Method," *International Symposium on Search-Based Software Engineering (SSBSE)*, Lecture Notes in Computer Science, vol. 9962, pp. 259–266, 2016, Springer. <https://eprints.whiterose.ac.uk/id/eprint/104204/1/c43.pdf>
- [36] AVMf GitHub Repository. [Online]. Available: <https://github.com/AVMf/avmf>
- [37] D. Humeniuk, F. Khomh și G. Antoniol, "AmbieGen: A search-based framework for autonomous systems testing," *Science of Computer Programming*, vol. 230, 2023. https://www.sciencedirect.com/science/article/pii/S0167642323000722?utm_source=chatgpt.com
- [38] AmbieGen GitHub Repository. [Online]. Available: <https://github.com/swat-lab-optimization/ambiegen> (pypi.org)
- [39] K. Claessen și J. Hughes, "QuickCheck: a lightweight tool for random testing of Haskell programs," in *Proceedings of the 5th ACM SIGPLAN International Conference on Functional Programming (ICFP '00)*, Montreal, Canada, 2000, pp. 268–279, <https://dl.acm.org/doi/epdf/10.1145/351240.351266>
- [40] *QuickCheck Library Documentation*. Hackage. <https://hackage.haskell.org/package/QuickCheck>
- [41] *QuickCheck: Automatic Testing* (online tutorial). <https://sulzmann.github.io/ProgrammingParadigms/lec-haskell-testing.html>
- [42] Graham Hutton, "Programming language semantics: It's easy as 1,2,3", <https://people.cs.nott.ac.uk/pszgmh/123.pdf>

[43] R. Ferdous, C.-k. Hung, F. M. Kifetew, D. Prandi, A. Susi, EvoMBT at the SBST 2022 Tool Competition, in Proceedings of the 15th IEEE/ACM International Symposium on Search-Based Software Engineering (SBST 2022 Tool Competition)

<https://www.researchgate.net/publication/368258770> EvoMBT at the SBST 2022 tool competition

[44] Ferdous, Raihana, Hung, Chia-kang, Kifetew, Fitsum Meshesha, Prandi, Davide, Susi, Angelo. (2023). EvoMBT: Evolutionary model based testing. Science of Computer Programming.

<https://www.researchgate.net/publication/369277723> EvoMBT Evolutionary Model Based Testing

[45] EvoMBT <https://github.com/iv4xr-project/iv4xr-mbt> + <https://github.com/iv4xr-project/iv4xr-mbt/wiki>

[46] SongYang Tan and Ming Fan. 2022. AdaFrenetic tool.

<https://github.com/TayYim/adafrenetic-sbst22>

[47] S. Yan and M. Fan, "AdaFrenetic at the SBST 2022 Tool Competition," 2022 IEEE/ACM 15th International Workshop on Search-Based Software Testing (SBST), <https://dl.acm.org/doi/epdf/10.1145/3526072.3527530>